

```
# TensorFlow and tf.keras
import tensorflow as tf
from tensorflow import keras
# Helper Libraries
import numpy as np
import matplotlib.pyplot as plt

fashion_mnist = keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) =
fashion_mnist.load_data()

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

train_images.shape

(60000, 28, 28)

len(train_labels)

60000

train_labels

array([9, 0, 0, ..., 3, 0, 5], shape=(60000,), dtype=uint8)

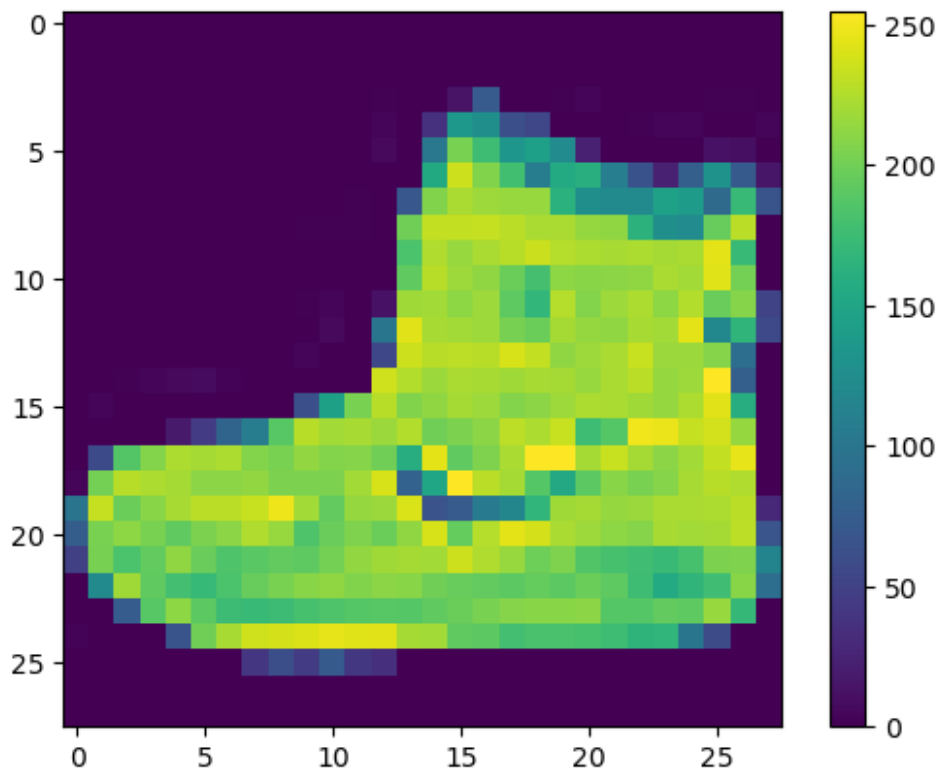
test_images.shape

(10000, 28, 28)

len(test_labels)

10000

plt.figure()
plt.imshow(train_images[0])
plt.colorbar()
plt.grid(False)
plt.show()
```



```
train_images = train_images / 255.0  
test_images = test_images / 255.0
```

```
plt.figure(figsize=(10,10))
```

```
for i in range(25):  
    plt.subplot(5, 5, i+1)  
    plt.xticks([])  
    plt.yticks([])  
    plt.grid(False)  
    plt.imshow(train_images[i], cmap=plt.cm.binary)  
    plt.xlabel(class_names[train_labels[i]])
```

```
plt.show()
```



```
model = keras.Sequential([
keras.layers.Flatten(input_shape=(28, 28)),
keras.layers.Dense(128, activation=tf.nn.relu),
keras.layers.Dense(10, activation=tf.nn.softmax)
])
```

C:\Users\DELL-PC\anaconda3\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

```
model.compile(optimizer='adam',
```

```
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

```
model.fit(train_images, train_labels, epochs=5)
```

Epoch 1/5

1875/1875 ————— 7s 3ms/step - accuracy: 0.8260 - loss: 0.4929

Epoch 2/5

1875/1875 ————— 8s 4ms/step - accuracy: 0.8641 - loss: 0.3749

Epoch 3/5

1875/1875 ————— 7s 4ms/step - accuracy: 0.8778 - loss: 0.3354

Epoch 4/5

1875/1875 ————— 6s 3ms/step - accuracy: 0.8844 - loss: 0.3113

Epoch 5/5

1875/1875 ————— 6s 3ms/step - accuracy: 0.8908 - loss: 0.2951

<keras.src.callbacks.history.History at 0x184e74cb230>

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
```

```
print('Test accuracy:', test_acc)
```

313/313 ————— 1s 2ms/step - accuracy: 0.8717 - loss: 0.3549

Test accuracy: 0.8716999888420105

```
predictions = model.predict(test_images)
```

313/313 ————— 1s 3ms/step

```
predictions[0]
```

```
array([1.5186270e-05, 3.4277403e-07, 3.9737174e-08, 3.1932519e-08,  
       6.7633357e-07, 5.6167669e-04, 5.9907035e-07, 5.9306689e-02,  
       3.9617251e-05, 9.4007510e-01], dtype=float32)
```

```
import numpy as np
```

```
np.argmax(predictions[0])
```

```
np.int64(9)
```

```
test_labels[0]
```

```
np.uint8(9)
```

```
def plot_image(i, predictions_array, true_label, img):
```

```

    predictions_array, true_label, img = predictions_array[i], true_label[i],
img[i]

    plt.grid(False)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(img, cmap=plt.cm.binary)

    predicted_label = np.argmax(predictions_array)

    if predicted_label == true_label:
        color = 'blue'
    else:
        color = 'red'

    plt.xlabel("{} {:2.0f}% ({})".format(
        class_names[predicted_label],
        100*np.max(predictions_array),
        class_names[true_label]),
        color=color
    )

def plot_value_array(i, predictions_array, true_label):
    predictions_array, true_label = predictions_array[i], true_label[i]

    plt.grid(False)
    plt.xticks([])
    plt.yticks([])

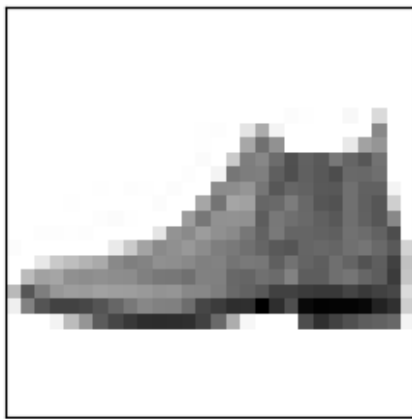
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])

    predicted_label = np.argmax(predictions_array)

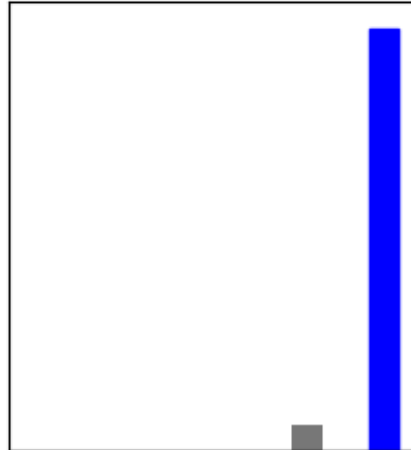
    thisplot[predicted_label].set_color('red')    # predicted
    thisplot[true_label].set_color('blue')        # actual

i = 0
plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plot_image(i, predictions, test_labels, test_images)
plt.subplot(1,2,2)
plot_value_array(i, predictions, test_labels)
plt.show()

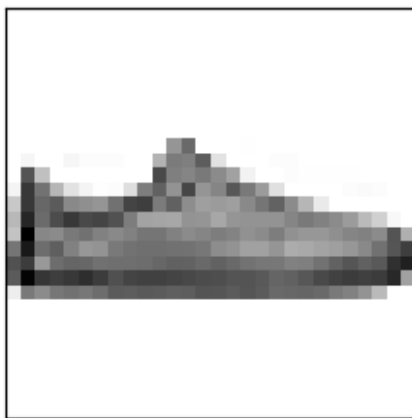
```



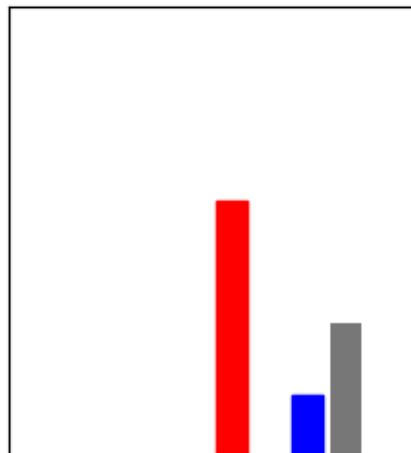
Ankle boot 94% (Ankle boot)



```
i = 12
plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plot_image(i, predictions, test_labels, test_images)
plt.subplot(1,2,2)
plot_value_array(i, predictions, test_labels)
plt.show()
```



Sandal 57% (Sneaker)



*# Plot the first X test images, their predicted label, and the true label*  
*# Color correct predictions in blue, incorrect predictions in red*

```
num_rows = 5
num_cols = 3
num_images = num_rows*num_cols
plt.figure(figsize=(2*2*num_cols, 2*num_rows))
for i in range(num_images):
    plt.subplot(num_rows, 2*num_cols, 2*i+1)
    plot_image(i, predictions, test_labels, test_images)
    plt.subplot(num_rows, 2*num_cols, 2*i+2)
```

```
plot_value_array(i, predictions, test_labels)
plt.show()
```

Cell In[24], line 8

```
plt.subplot(num_rows, 2*num_cols, 2*i+1)
^
```

IndentationError: expected an indented block after 'for' statement on line 7

```
# Grab an image from the test dataset
```

```
img = test_images[0]
```

```
print(img.shape)
```

```
# Add the image to a batch where it's the only member.
```

```
img = (np.expand_dims(img,0))
```

```
print(img.shape)
```

```
predictions_single = model.predict(img)
```

```
print(predictions_single)
```

```
plot_value_array(0, predictions_single, test_labels)
```

```
_ = plt.xticks(range(10), class_names, rotation=45)
```

```
np.argmax(predictions_single[0])
```